

المحاضرة 10

المصفوفات (2) : Arrays

د. سهيل الحمود

د. أسامه ناصر

2025-2026

نسخة تفاعلية من المحاضرات

<https://ossnass.github.io/Programmin-1>



ترتيب المصفوفات Sorting

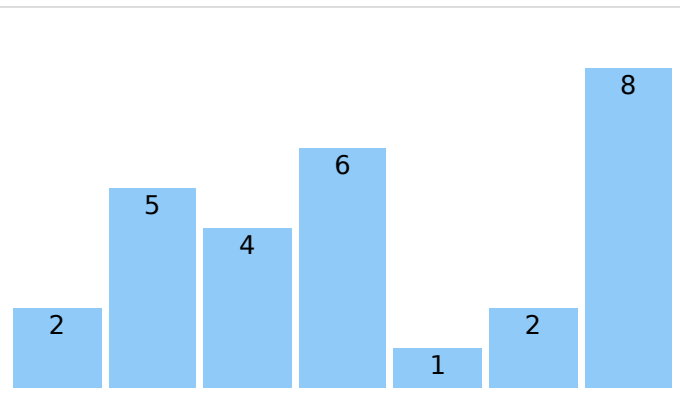
- من أهم المفاهيم الموجودة ضمن المصفوفات هي ترتيب القيم ضمنها
- تصاعدياً أو تنازلياً وتعتبر قضية أساسية
- تتواجد لدينا مجموعة كبير من خوارزميات الترتيب منها الخوارزميات الحقيقية:
 - Bubble sort الفرز الفقاعي
 - Selection Sort الفرز بالاختيار
 - Insertion Sort الفرز بالإدخال
 - Quick Sort الفرز السريع
 - Merge Sort الفرز بالدمج
- خوارزميات فرز موجودة لكن غير مستخدمة (لأنها تعتبر Meme)
 - Stalin Sort فرز ستالين (نعم ستالين ماغيره رئيس الاتحاد السوفيتي)
 - Quantum Sort الفرز الكومي
 - Random Sort الفرز العشوائي

الفرز الفقاعي Bubble Sort

Bubble Sort Visualizer

السابق التالي إعادة للبداية

```
1 procedure bubbleSort(A):  
2   n = length(A)  
3   for i from 0 to n-1:  
4     for j from 0 to n-i-1:  
5       if A[j] > A[j+1]:  
6         swap A[j], A[j+1]
```



الخطوة 1 / 72

تهيئة المصفوفة

الفرز الفقاعي Bubble Sort

- من مشاكل الفرز الفقاعي:
- البطء إذ أنها تتطلب المرور على المصفوفة كاملة مرتين للتحقق من صحة التنفيذ (التفسير في مقرر الخوارزميات 1)
- أما محاسنها:
- سهولة التحقيق والتنفيذ

الفرز الفقاعي Bubble Sort

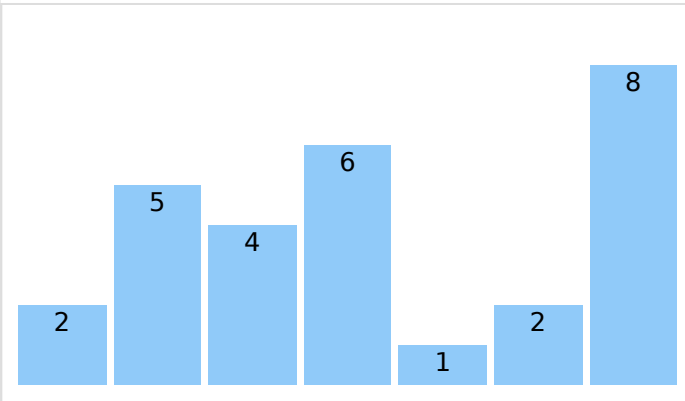
```
void bubbleSort(int arr[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        for (int j = 0; j < n - i - 1; j++) {  
            if (arr[j] > arr[j + 1]) {  
                int temp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = temp;  
            }  
        }  
    }  
}
```

الفرز بالاختيار Selection Sort

Selection Sort Visualizer

السابق التالي إعادة للبداية

```
1 procedure selectionSort(A):  
2     n = length(A)  
3     for i from 0 to n-2:  
4         minIndex = i  
5         for j from i+1 to n-1:  
6             if A[j] < A[minIndex]:  
7                 minIndex = j  
8         swap A[i], A[minIndex]
```



الخطوة 1 / 70

تهيئة المصفوفة

الفرز بالاختيار Selection Sort

- من مشاكل الفرز بالاختيار:
- البطء إذ أنها تتطلب المرور على المصفوفة كاملة مرتين للتحقق من صحة التنفيذ (التفسير في مقرر الخوارزميات 1)
- على الرغم من ذلك تعتبر أسرع من الفرز الفقاعي
- أما محاسنها:
- سهولة التحقيق والتنفيذ

الفرز الاختيار Selection Sort

```
void selectionSort(int arr[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        int minIndex = i;  
        for (int j = i + 1; j < n; j++) {  
            if (arr[j] < arr[minIndex]) {  
                minIndex = j;  
            }  
        }  
        if (minIndex != i) {  
            swap(arr[i], arr[minIndex]);  
        }  
    }  
}
```

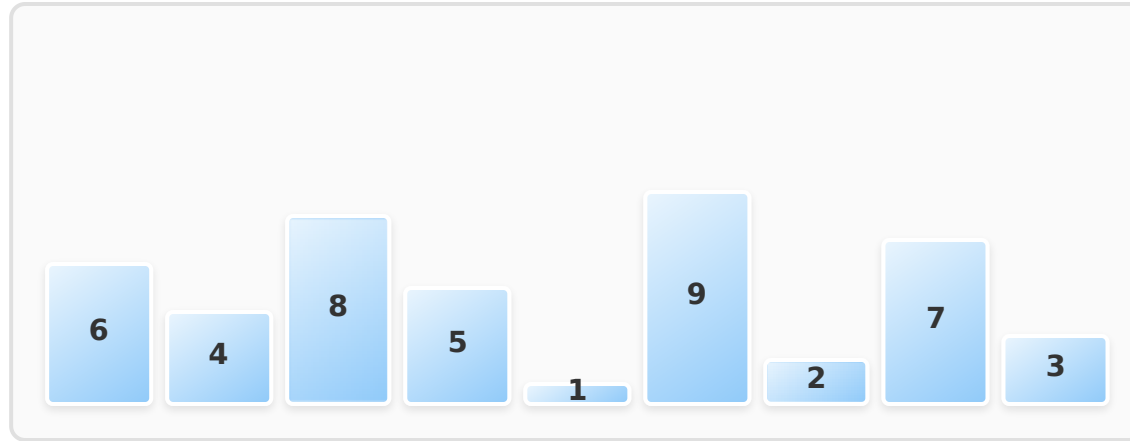

البحث عن عناصر ضمن المصفوفة

البحث الخطي

Linear Search Visualizer

بحث

البحث عن: e.g. 4



البحث عن عناصر ضمن المصفوفة

البحث الخطي

```
int linearSearch(int [] array, int n , int value){  
    for(int i=0;i<n;i++){  
        if(array[i]==value)  
            return i;  
    }  
    return -1;  
}
```

البحث عن عناصر ضمن المصفوفة

البحث الثنائي

- هذه الخوارزمية أسرع بكثير من البحث الخطي
- نقوم بقراءة العنصر المتواجد في منتصف المصفوفة
- في حال كان مساوياً للعنصر الذي نريده، نتوقف
- في حال كان أصغر من القيمة نبحث في الفرع الأيسر (من بداية المصفوفة حتى العنصر الذي يسبق المنتصف)
- في حال كان أكبر من القيمة نبحث في الفرع الأيمن (من العنصر التالي للعنصر الواقع في منتصف المصفوفة حتى نهاية المصفوفة)
- عملية البحث تكون بتطبيق نفس العملية (البحث الثنائي) على النصف المحدد
- في حال وصلنا لمصفوفة فارغة نعتبر العنصر غير موجود

البحث عن عناصر ضمن المصفوفة

البحث الثنائي

Binary Search Visualizer

العودة للبداية

التالي

السابق

البحث

at (10-60)

50

42

35

30

22

16

12

فهرس أول عنصر: N/A | فهرس المنتصف: N/A | فهرس آخر عنصر: N/A

البحث عن عناصر ضمن المصفوفة

البحث الثنائي

- يتميز البحث الثنائي بالسرعة العالية جداً
- لذلك يستخدم بكثرة في عمليات البحث
- المشكلة الأساسية ضمان إبقاء المصفوفة مرتبة مع تغير العناصر (لذلك نحتاج لخوارزمية ترتيب سريعة)
- حل آخر ببناء ما يعرف بالشجرة الثنائية (مقرر الخوارزميات 2)

البحث عن عناصر ضمن المصفوفة

البحث الثنائي النسخة التكرارية

```
int binarySearch(int arr[], int n, int target) {  
    int low = 0;    int high = n - 1;  
    while (low ≤ high) {  
        int mid = low + (high - low) / 2;  
        if (arr[mid] == target) {  
            return mid;  
        } else if (arr[mid] < target) {  
            low = mid + 1;  
        } else {  
            high = mid - 1;  
        }  
    }  
    return -1;  
}
```


البحث عن عناصر ضمن المصفوفة

البحث الثنائي النسخة العودية

```
int bsr(int arr[], int low, int high, int target) {  
    if (low > high) {return -1;}  
    int mid = low + (high - low) / 2;  
    if (arr[mid] == target) {return mid;}  
    else if (arr[mid] < target) {  
        return bsr(arr, mid + 1, high, target);  
    }else {  
        return bsr(arr, low, mid - 1, target);  
    }  
}
```

هل المصفوفات ببعد واحد فقط؟

- لنعرف الإجابة على هذا السؤال سندرس كيفية تمثيل الصور في الحاسب
- الصورة الرمادية عبارة عن مصفوفة ثنائية البعد
- في كل نقطة (تقاطع أسطر i مع أعمدة j نتواجد لدينا قيمة لون)

$image[0][0] = 255$
 $image[180][174] = 0$



المصفوفات متعددة الأبعاد

- يتم تعريف هذه المصفوفات باستخدام أقواس إضافية

```
int twoD[4][2];  
int threeD[9][5][4];  
int fourD[10][99][203][900];  
twoD[2][3]=0;  
fourD[3][33][20][200]=threeD[1][3][2]
```

أكثر خطأ مصفوفة شيوعاً

- ماذا يمكن أن يحدث في حال حاولنا الوصول لعنصر غير موجود في المصفوفة

```
int x[]={1,2,34,4};  
cout<<x[77];
```

- يطبع قيم عشوائية بفعل سلوك غير محدد وغير مفهوم
- في لغات برمجة أخرى يتم رمي خطأ throw an exception للوصول لعنصر خارج مجال المصفوفة